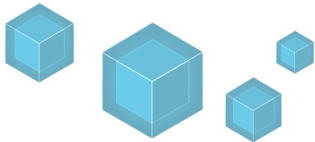
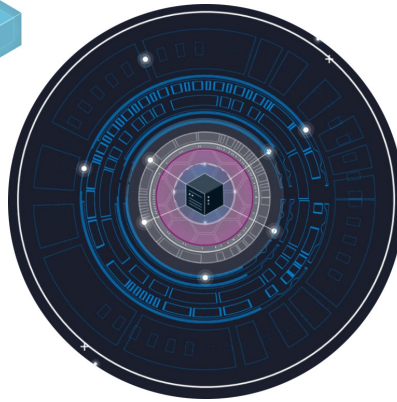


Unikernels: The Next Big Thing?



This article discusses the benefits of unikernels and how they can solve the security and management issues that arise with running modern applications in containers. It also describes how to build a Nanos based Node.js unikernel instance running a pocket calculator app on Amazon Web Services.



Technology continues to evolve each day. Cloud computing, too, has evolved from virtual machines to containers. Unikernels may well be the next step in this evolutionary process to help solve the challenges posed by containers.

Modern applications are scalable and modular. They require rapid setups (deployments) and on-demand provisioning of resources. Cloud based virtual machines (VMs) with their elasticity and ease of setup were ideal for running these applications until cost and portability became an issue. Containers, a technology born in 1979, came to the rescue after 34 years of evolution. They mitigated most of the issues related to portability and rapid setups, and also supported the modular approach of modern applications without any cost implications because in most cases they were being deployed

on a single VM or host. Containers introduced an additional layer of virtualisation on top of VMs. This resulted in the gradual replacement of VMs based application setups, and containers soon became the solution of choice for running modern applications.

However, containers add a few complexities to the system as they require a full-fledged operating system (OS) to function and add a layer of software on top of the operating system. This increases the number of moving parts within the system that require maintenance and thus the points of failure, enhancing the chances of security breaches.

Let's quickly look at a real-world scenario. Think of a simple standalone website developed in Java, running from a Docker container and hosted on the cloud. Docker itself runs on a VM, which has an operating system like Linux, and this VM runs on another specialised software known as bare-metal hypervisor, which runs on top of a physical machine. As an example, this can be XenServer or a full-scale

operating system like Linux as well. So here, a technical team will have to manage Docker configurations and ensure that the Docker image has no security issues. Next, it will have to maintain and secure the guest operating system on top of which Docker is installed and running. It will also have to ensure all the configurations related to security are in place and perform health checks of the VM. Because if this VM fails, Docker will also fail. Additionally, it will need to maintain the hypervisor or the Linux host and ensure its security, because if someone gains access to this host they will have access to every VM running on it. Health checks and monitoring need to be performed for these as well, because if this host fails, all the VMs will fail and impact services.

Ideally, in a cloud environment, as end users we don't need to manage this hypervisor or host as it is the responsibility of the cloud provider. However, it is a point of failure. So, we have identified three different points of failures and gaps in security that need to be managed and monitored, requiring a considerable amount of effort. This is where unikernels excel; they address all these issues and concerns by minimising the moving parts or layers of software, thus reducing the points of failure along with security gaps.

We will have more clarity on this once we begin our journey with unikernels — understand what they are, what advantages they have, and how to build them. So let's get started.

What is a unikernel?

In simple terms, a unikernel allows an application to run like an operating system itself without depending on an actual OS for execution. Once this unikernel is compiled or built, it cannot be changed or modified. As per the 'Unikernels: Library Operating Systems for the Cloud' research paper, unikernels are single-purpose appliances that are compile-time